



Programming Language 2 _CS 111T

Project

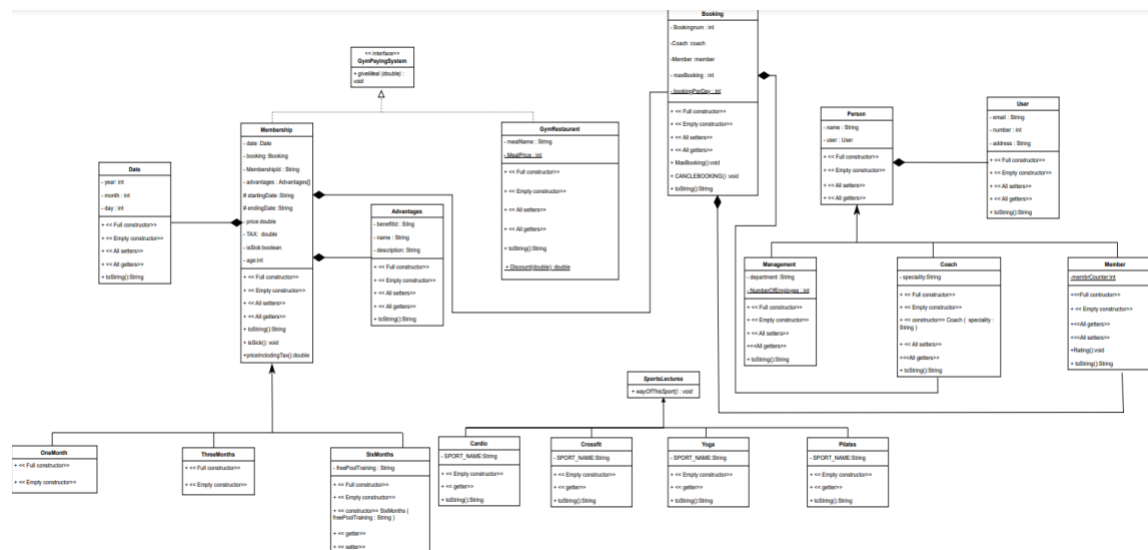
GYM SYSTEM

21y

I Project overview

The project is about a Gym System that makes it easier to handle reservations and new memberships in fitness facilities. It has features for adding, removing, and displaying memberships or bookings, registering new members, scheduling training sessions with coaches, and placing restaurant orders at the gym. The system's goals are to increase user satisfaction and expedite the gyms' everyday operations.

II Project class diagram



1. Class Advantages :

- Three private variables** ("benefitId", "name", and "description") are declared within the class.

-Constructors: Two constructors are provided: The first constructor takes three parameters (benefitId, name, description) and assigns their values to the respective variables.-The second constructor sets default values for the variables.

- **Setters & Getters:** - Setter and getter methods are created for each variable.

- **Override toString Method:** The toString() method is overridden to return a string containing the values of the variables in a specific format.

2. Class Booking:

- **Private variables** "Bookingnum", "coach", "member", "maxBooking", and "bookingPerDay" are declared within the class.
- "bookingPerDay" is declared as **static** to keep track of the total bookings made per day.

-Constructors:- Two constructors are provided: - The first constructor takes parameters for booking number, coach, member, and maximum booking allowed per day. It initializes the instance variables with the provided values and increments the static variable "bookingPerDay". - The second constructor sets default values for the variables.

-Setters & Getters: Setter and getter methods are created for each variable.

- **Methods:-** MaxBooking(): Checks if the current number of bookings exceeds the maximum allowed bookings per day and displays a message if there are no available bookings. - cancelBooking(): Decrements the static variable "bookingPerDay" when a booking is canceled and prints a message indicating the cancellation.

- **Override toString Method:-** The toString() method is overridden to return a string representation of the booking object, including details about the coach, member, and maximum bookings.

3. Class Cardio

- The class "Cardio" **extends** the class "SportsLecturs", indicating that it inherits from it.

-Constant Variable: - A final variable "SPORT_NAME" is declared and assigned the value "cardio".

- **Override toString Method:-** The toString() method is overridden to return a string representation of the "Cardio" object, including the value of the "SPORT_NAME" variable.

- **Override wayOfThisSport Method:** - The abstract method "wayOfThisSport()" from the superclass "SportsLecturs" is overridden in the "Cardio" class.

4. Class Coach:

- The class "Coach" is defined, indicating that it **extends** the class "person".

- **Variables:** - It has a private instance variable named "speciality" to store the coach's specialty.

. **Constructors:** - The class has three constructors: - The first constructor takes three parameters: "speciality", "name", and "user". It initializes the superclass "person" with the "name" and "user" parameters, and sets the "speciality" variable. - The second constructor initializes all fields with default values. It calls the first constructor with empty strings and null for the "name" and "user" parameters. - The third constructor takes only the "speciality" parameter and sets the "speciality" variable.

- **Setters & Getters:** - It has setter and getter methods for the "speciality" variable to set and retrieve the coach's specialty.

5. Class Date:

- The class "Date" is declared as **final**, indicating that it cannot be subclassed or extended.

Variables:- It has three private instance variables: "year", "month", and "day" to represent the components of a date.

Constructors: - Two constructors are provided: - The first constructor takes parameters for the year, month, and day, and initializes the instance variables using the setter methods. - The second constructor initializes all fields with default values (zeros for year, month, and day).

. **Setters & Getters:-** Setter methods are provided for each variable to set their values with some validation checks: - The setYear method checks if the year is greater than or equal to 2024. - The setMonth method checks if the month is between 1 and 12. - The setDay method checks if the day is between 1 and 31.

. **Override toString Method:** - The toString() method is overridden to return a string representation of the date in the format "year/month/day".

6. Class GymPayingSystem:

- The **interface** "GymPayingSystem" is declared.

Method:- It declares a single method named "giveMeal" with a parameter of type double representing the weight.

7. Class GymRestaurant:

This class represents a gym restaurant that **implements** the "GymPayingSystem" interface to provide meals to gym members based on their weight. It also provides methods to get and set meal details and calculate discounts based on weight.

. **Variables:-** It has two private instance variables: "mealName" to store the name of the meal and "Mealprice" as a static variable to store the price of the meal.

. **Constructors:** - Two constructors are provided: - The first constructor takes a parameter for the meal name and initializes the "mealName" variable. - The second constructor initializes the "mealName" variable with an empty string using the first constructor.

. **Override Method:-** The class implements the "giveMeal" method from the "GymPayingSystem" interface. It checks if the weight provided is greater than or equal to 100kg and prints a message accordingly.

. **Override toString Method:-** The toString() method is overridden to return a string representation of the "GymRestaurant" object, including the meal name.

. **Static Methods:** - There's a static method named "Discount" that calculates and returns a discount based on the weight provided.

. **Static Variable Getter and Setter:** - Getter and setter methods are provided for the static "Mealprice" variable.

8. Class Management:

- "Management" **extends** the class "person" and represents management personnel

. **Variables:**- It has two private instance variables: "department" to store the department of the management personnel and "NumOfEmployee" as a **static** variable to count the number of employees.

. **Constructors:**- Two constructors are provided: - The first constructor takes parameters for the department, name, and user. It initializes the superclass "person" with the name and user, sets the department, and increments the **static** variable "NumOfEmployee". - The second constructor initializes all fields with default values (empty strings and null).

. **Setters & Getters:** - Setter and getter methods are provided for the "department" variable.

. **Override toString Method:** - The toString() method is overridden to return a string representation of the "Management" object, including the department.

Static Variable Getter: - A static getter method is provided to retrieve the number of employees.

9. Class Member:

- The class "Member" is defined, indicating that it **extends** the class "person".

. **Variables:** - It has a private **static** variable "memberCounter" to keep track of the number of members.

. **Constructors:** - Two constructors are provided: - The first constructor takes parameters for the member's name and user details. It initializes the superclass "person" with the provided values, increments the "memberCounter", and prints the current number of members. - The second constructor initializes the member's name with an empty string and the user details with null.

. **Static Method:**- A static method named "getMemberCounter()" is provided to retrieve the current number of members.

. **Method:** - The method "Rating()" is provided to determine the rating based on the number of members. If the number of members exceeds 100, the rating is set to 4.5; otherwise, it's below 4.5.

10. Class Membership:

"Membership" that implements the interface "GymPayingSystem".

. **Variables:**- It has several instance variables including "date", "booking", "MembershipId", "advantages", "startingDate", "endingDate", "price", "TAX", "isSick", and "age".

. **Constructors:** - Two constructors are provided: - The first constructor takes parameters for all instance variables and initializes them accordingly - The second constructor initializes all fields with default values.

. **Override Method:**- The class implements the "giveMeal" method from the "GymPayingSystem" interface, but it doesn't provide any implementation for it.

. **Getter and Setter Methods:**

- Getter and setter methods are provided for instance variables.

Methods:

- The method "isSick()" checks if the member is sick and prints a corresponding message.

- The method "PriceIncludingTax()" calculates the price including tax based on the age of the member.

- The method "getAdvantage()" returns a string containing the advantages of the membership.

. **Override toString Method:**

- The toString() method is overridden to return a string representation of the "Membership" object, including all instance variables.

11. Class OneMonth:

- The class "OneMonth" is defined, indicating that it **extends** the class "Membership".

Constructors: - Two constructors are provided: - The first constructor takes parameters for all instance variables inherited from the "Membership" class and initializes them accordingly by calling the superclass constructor. - The second constructor is a default constructor with no parameters.

Override Method: - The class overrides the "giveMeal" method inherited from the superclass "Membership". It provides an implementation specific to a one-month membership, which simply prints a message indicating that the member has one free meal.

12. Class ReadText:

Instance Variables: - It has a private instance variable "input" of type Scanner for reading input from the file.

. Methods:

- openTextFile(String fileName): Opens the specified text file for reading. If the file is not found, it catches a FileNotFoundException and prints an error message.

- readFromFile(): Reads text from the opened file line by line using a Scanner object. It catches NoSuchElementException if the file is improperly formed, prints an error message, and exits the program. It also catches IllegalStateException if there's an error reading from the file and prints an error message.

- closeFile(): Closes the file if it was opened.

This class provides methods to open a text file, read from it, and close it properly. **It handles exceptions that may occur during file operations to ensure graceful error handling.**

13. Class SixMonths:

- The class "SixMonths" is defined, indicating that it **extends** the class "Membership".

. Instance Variable:

- It has a private instance variable "freePoolTraining" to store information about free pool training.

. Constructors:

- Two constructors are provided:

- The first constructor takes parameters for all instance variables inherited from the "Membership" class as well as the "freePoolTraining" variable, and initializes them accordingly by calling the superclass constructor. It also initializes the "freePoolTraining" variable.

- The second constructor takes only the "freePoolTraining" variable as a parameter and initializes it.

. Override Method: - The class overrides the "giveMeal" method inherited from the superclass "Membership". It provides an implementation specific to a six-month membership, which prints a message indicating that the member has six free meals.

. Getter and Setter Methods:

- Getter and setter methods are provided for the "freePoolTraining" variable.

14. Class SportsLecturs:

Abstract Class Definition:

- The class "SportsLecturs" is defined as abstract, indicating that it cannot be instantiated directly and may contain abstract methods.

. Abstract Method:

- It has one abstract method named "wayOfThisSport()". Abstract methods are declared without implementation and must be implemented by non-abstract subclasses.

15. Class ThreeMonths:

- The class "ThreeMonths" is defined, indicating that it **extends** the class "Membership".

. **Constructors:** - Two constructors are provided: - The first constructor takes parameters for all instance variables inherited from the "Membership" class and initializes them accordingly by calling the superclass constructor. - The second constructor is a default constructor with no parameters.

. **Override Method:-** The class overrides the "giveMeal" method inherited from the superclass "Membership". It provides an implementation specific to a three-month membership, which prints a message indicating that the member has three free meals.

16. Class User :

. **Variables:**

- It has three private instance variables: "email" to store the user's email address, "number" to store the user's phone number, and "address" to store the user's address.

. **Constructors:**

- Two constructors are provided:

- The first constructor takes parameters for the email, number, and address, and initializes the instance variables accordingly.

- The second constructor initializes all fields with default values (empty string for email and address, and zero for number).

. **Override toString Method:**

- The toString() method is overridden to return a string representation of the "User" object, including the email, number, and address

17. Class WriteText:

. **Instance Variable:**

- It has a private instance variable named "output" of type Formatter. This variable is used to write formatted text to the file.

. **Method: openTextFile:**

- This method takes a fileName parameter and attempts to open/create the file for writing.

- It catches SecurityException and FileNotFoundException exceptions that might occur during the file operation and prints corresponding error messages.

. **Method: writeToFile:**

- This method takes a Membership object as a parameter and writes its string representation to the file.

- It catches a FormatterClosedException that might occur if the output formatter is closed before writing.

. **Method: closeFile:** - This method closes the file if it's open. - It checks if the output formatter is not null before attempting to close it.

18. Class Yoga:

-"Yoga," **extends** the abstract class "SportsLecturs

. **Constant Variable:**

- It has a final constant variable "SPORT_NAME" set to "yoga" to represent the name of the sport.

. **Constructor:**

- The class has a default constructor that initializes an object of type "Yoga."

. **Override toString Method:**

- The toString() method is overridden to return a string representation of the "Yoga" object, including the sport name.

. Override wayOfThisSport Method:

- The abstract method "wayOfThisSport()" from the superclass "SportsLecturs" is overridden in the "Yoga" class.
- This method provides specific implementation details for the sport of yoga, including a description of how it is practiced through meditation and breathing techniques.

19. Class Person :

. Variables:

- It has two private instance variables: "name" to store the person's name and "user" to store user-related information.

. Constructor:

- The class has two constructors:
- The first constructor takes parameters for the person's name and user details and initializes them accordingly.
- The second constructor initializes the name with an empty string and the user with null, using the first constructor.

. Getter and Setter Methods:

- Getter and setter methods are provided for both instance variables to set and get their values.

20. Class pilates:

. Constant Variable:

- It has a final constant variable "SPORT_NAME" set to "pilates" to represent the name of the sport.

. Override toString Method:

- The toString() method is overridden to return a string representation of the "pilates" object, including the sport name.

. Override wayOfThisSport Method:

- The abstract method "wayOfThisSport()" from the superclass "SportsLecturs" is overridden in the "pilates" class.
- This method provides specific implementation details for the sport of Pilates, explaining its focus on equalizing body sides and balancing motion and strength.

III Concepts covered

III.1 Interface

```
/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
 * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit this template
 */
package newproject;
interface GymPayingSystem {
    public abstract void giveMeal(double weight);
}
```

III.2 Inheritance

```

1  /**
2   * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
3   * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit this template
4   */
5   package newproject;
6
7   public class Cardio extends SportsLecturs{
8
9
10      private final String SPORT_NAME = "cardio";
11
12      public Cardio() {
13      }
14
15      public String getSPORT_NAME() {
16          return SPORT_NAME;
17      }
18
19      @Override
20      public String toString() {
21          return "Cardio[" + "SPORT_NAME=" + SPORT_NAME + ']';
22      }
23
24      @Override
25      public void wayOfThisSport() {
26          System.out.println("Cardio starts with a warmup and body movements, short for cardiovascular exercise, referring to activities that elevate your heart rate.");
27      }
28
29  }
30

```

```

1  /**
2   * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit this template
3   */
4   package newproject;
5
6   public class pilates extends SportsLecturs {
7
8      private final String SPORT_NAME = "pilates";
9
10      public pilates() {
11      }
12
13      public String getSPORT_NAME() {
14          return SPORT_NAME;
15      }
16
17      @Override
18      public String toString() {
19          return "Pilates[" + "SPORT_NAME=" + SPORT_NAME + ']';
20      }
21
22      @Override
23      public void wayOfThisSport() {
24          System.out.println("Pilates works by focusing on equalizing body sides of the body and balancing motion and strength.");
25      }
26
27  }
28

```

```

2  * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change the license of the project.
3  * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit this template.
4  */
5  package newproject;
6
7      public class Coach extends person {
8
9          private String speciality;
10
11      public Coach(String speciality, String name, User user) {
12          super(name, user);
13          this.speciality = speciality;
14      }
15      public Coach() {
16          this( speciality: "", name: "", user: null);
17      }
18
19      public Coach(String speciality) {
20          this.speciality = speciality;
21      }
22
23      public void setSpeciality(String speciality) {
24          this.speciality = speciality;
25      }
26
27      public String getSpeciality() {
28          return speciality;
29      }
30
31      @Override
32      public String toString() {
33          return "Coach[" + "speciality=" + speciality + ']';
34      }
35  }
36
37  package newproject;
38
39      public class Management extends person {
40
41          private String department;
42          private static int NumOfEmployee;
43
44      public Management(String department, String name, User user) {
45          super(name, user);
46          this.department = department;
47          NumOfEmployee++;
48          System.out.println("The number of Employees: " + NumOfEmployee);
49      }
50
51      public Management() {
52          this( department: "", name: "", user: null);
53      }
54
55      public String getDepartment() {
56          return department;
57      }
58
59      public void setDepartment(String department) {
60          this.department = department;
61      }
62
63      @Override
64      public String toString() {
65          return "Management[" + "department=" + department + ']';
66      }
67
68      public static int getNumOfEmployee() {
69          return NumOfEmployee;
70      }
71  }

```

```

object {

  Cardio extends SportsLectures {

    > final String SPORT_NAME = "cardio";

    rdio() {

    ring getSPORT_NAME() {
    > SPORT_NAME;

    ride
    > String toString() {
    > return "Cardio[" + "SPORT_NAME=" + SPORT_NAME + "']";

    ride
    > void wayOfThisSport() {
    > system.out.println("Cardio starts with a warmup and body movements, short for cardiovascular exercise, referring to activities that elevate your heart rate.");

  }

  > Membership {

  > >, Booking booking, String MembershipId, Advantages[] advantages, String startingDate, String endingDate, double price, boolean issick, int age) {
  > MembershipId, advantages, startingDate, endingDate, price, issick, age);

  > >le weight) {
  > "you have 1 free Meal..";

```

```

\
/* @author nouira
 */
public class OneMonth extends Membership {

    public OneMonth(Date date, Booking booking, String MembershipId, Advantages[] advantages, String startingDate, String endingDate, double price,
        super(date, booking, MembershipId, advantages, startingDate, endingDate, price, isSick, age);
    }

    public OneMonth() {
    }

    @Override
    public void giveMeal(double weight) {
        System.out.println(x: "you have 1 free Meal..");
    }
}

10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
Membership {
    public Membership(Date date, Booking booking, String MembershipId, Advantages[] advantages, String startingDate, String endingDate, double price, boolean isSick, int age) {
        super(date, booking, MembershipId, advantages, startingDate, endingDate, price, isSick, age);
    }

    public Membership() {
    }

    @Override
    public void giveMeal(double weight) {
        System.out.println(x: "you have 3 free Meals..");
    }
}

/*
public class ThreeMonths extends Membership {

    public ThreeMonths(Date date, Booking booking, String MembershipId, Advantages[] advantages, String startingDate, String endingDate, double p:
        super(date, booking, MembershipId, advantages, startingDate, endingDate, price, isSick, age);
    }

    public ThreeMonths() {
    }

    @Override
    public void giveMeal(double weight) {
        System.out.println(x: "you have 3 free Meals..");
    }
}

```

```

1 public class SixMonths extends Membership {
2
3     private String freePoolTraining;
4
5     public SixMonths(String freePoolTraining, Date date, Booking booking, String MembershipId, Advantages[] advantages, String startingDate, String endingDate, double price, boolean isSick, int age) {
6         super(date, booking, MembershipId, advantages, startingDate, endingDate, price, isSick, age);
7         this.freePoolTraining = freePoolTraining;
8     }
9
10
11
12     public SixMonths(String freePoolTraining) {
13         this.freePoolTraining = freePoolTraining;
14     }
15     public SixMonths() {
16
17     }
18
19
20
21     @Override
22     public void giveMeal(double weight) {
23         System.out.println("you have 6 free Meals..");
24     }
25
26
27     public String getFreePoolTraining() {
28         System.out.println("you have free pool training");
29     }
30 }

```

```

    }, Booking booking, String MembershipId, Advantages[] advantages, String startingDate, String endingDate, double price, boolean isSick, int age)
    startingDate, endingDate, price, isSick, age);

```

```

    ,
    public SixMonths() {

    }

    @Override
    public void giveMeal(double weight) {
        System.out.println("you have 6 free Meals..");
    }

    public String getFreePoolTraining() {
        System.out.println("you have free pool training");
        return freePoolTraining;
    }

    public void setFreePoolTraining(String freePoolTraining) {
        this.freePoolTraining = freePoolTraining;
    }

```

III.3 Abstract classes

```
1  /*
2   * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to view and
3   * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit
4   */
5   package newproject;
6
7   ④ abstract class SportsLecturs {
8
9
10
11   ④     public abstract void wayOfThisSport();
12
13   }
```

III.4 Polymorphism

```
case 2:
system.out.print(=: "Enter membership id to remove: ");
String membId = input.next();
boolean removed = false;
for (int i = 0; i < listt.size(); i++) {
    Membership ele = listt.get(index: i);
    if (ele.getMembershipId().equals(anObject: membId)) {
        listt.remove(index: i);
        removed = true;
        break;
    }
}
if (removed) {
    System.out.println("Membership with id " + membId + " is deletd");
} else {
    System.out.println("there is no Membership with id " + membId + " exist");
}
break;

case 3:
for (GymPayingSystem ele : list) {
    // (Polymorphism)
    System.out.println(x: ele);
    system.out.printf(format: "Membership meal: %.2f%n",);
    ele.giveMeal(weight: 0);

    // Downcasting
    System.out.println(x: "-----");
    if (ele instanceof SixMonths) {
        System.out.println(x: ((SixMonths) ele).getFreePoolTraining()); //add an instance in 6 months class or method and call it here
    }
    System.out.println(x: "-----");
}
break;
```

III.5 Composition

```
package newproject;

public class person {

    private String name;
    private User user;

    public person(String name, User user) {
        this.name = name;
        this.user = user;
    }

    public person() {
        this(name="", user=null);
    }

    public User getUser() {
        return user;
    }

    public void setUser(User user) {
        this.user = user;
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }
}

package newproject;

public class Membership implements GymPayingSystem {

    private Date date;
    private Booking booking;
    private String MembershipId;
    private Advantages[] advantages;
    protected String startingDate;
    protected String endingDate;
    private double price;
    private final double TAX = 0.15;
    private boolean isSick;
    private int age;

    public Membership(Date date, Booking booking, String MembershipId, Advantages[] advantages, String startingDate, String endingDate, double price, boolean isSick, int age) {
        this.date = date;
        this.booking = booking;
        this.MembershipId = MembershipId;
        this.advantages = advantages;
        this.startingDate = startingDate;
        this.endingDate = endingDate;
        this.price = price;
        this.isSick = isSick;
        this.age = age;
    }

    public void MaxBooking() {
        if (bookingPerDay > maxBooking) {
            System.out.println("Sorry there is no Available rooms");
        }
    }

    public final void cancelBooking(){
        bookingPerDay--;
        System.out.println("booking is canceled");
    }

    public int getMaxBooking() {
        return maxBooking;
    }

    public void setMaxBooking(int maxBooking) {
        this.maxBooking = maxBooking;
    }

    public int getBookingPerDay() {
        return bookingPerDay;
    }

    public void setBookingPerDay(int bookingPerDay) {
        this.bookingPerDay = bookingPerDay;
    }

    public Coach getCoach() {
        return coach;
    }
}
```

```

1 package org.javaprojects;
2
3 public class person {
4
5     private String name;
6     private User user;
7
8     public person(String name, User user) {
9         this.name = name;
10        this.user = user;
11    }
12
13    public person() {
14        this.name = "", user=null;
15    }
16
17    public User getUser() {
18        return user;
19    }
20
21    public void setUser(User user) {
22        this.user = user;
23    }
24
25    public String getName() {
26        return name;
27    }
28
29    public void setName(String name) {
30        this.name = name;
31    }
32
33 }

```

III.6 ArrayList (in the main)

```
public class Main {  
  
    static ArrayList<GymPayingSystem> list = new ArrayList<>();  
    static ArrayList<Membership> listt = new ArrayList<>();  
    static ArrayList<Booking> list3 = new ArrayList<>();  
}
```

III.7 Array (with composition)

```
public static void fillList() {
    Advantages[] Advantages1 = new Advantages[2];
    Advantages1[0] = new Advantages(benefitId: "1", name: "Access to Premium Equipment", description: "Equipment");
    Advantages1[1] = new Advantages(benefitId: "2", name: "Personalized Training Sessions", description: "Training");
    OneMonth one = new OneMonth(date: null, booking: null, MembershipId: "", advantages: Advantages1, startingDate: "", endingDate: "", price: 1222.0, isSick: true, ag

    Advantages[] Advantages2 = new Advantages[2];
    Advantages2[0] = new Advantages(benefitId: "1", name: "Access to Premium Equipment", description: "Equipment");
    Advantages2[1] = new Advantages(benefitId: "2", name: "Personalized Training Sessions", description: "Training");
    OneMonth one1 = new OneMonth(date: null, booking: null, MembershipId: "", advantages: Advantages2, startingDate: "", endingDate: "", price: 1276.0, isSick: true, a

    Advantages[] Advantages3 = new Advantages[2];
    Advantages3[0] = new Advantages(benefitId: "1", name: "Access to Premium Equipment", description: "Equipment");
    Advantages3[1] = new Advantages(benefitId: "2", name: "Personalized Training Sessions", description: "Training");
    ThreeMonths three = new ThreeMonths(date: null, booking: null, MembershipId: "", advantages: Advantages3, startingDate: "", endingDate: "", price: 1900.0, isSick:

    Advantages[] Advantages4 = new Advantages[2];
    Advantages4[0] = new Advantages(benefitId: "1", name: "Access to Premium Equipment", description: "Equipment");
    Advantages4[1] = new Advantages(benefitId: "2", name: "Personalized Training Sessions", description: "Training");
    //ThreeMonths three1 = new ThreeMonths(Advantages4, null, "", "", 12.0, true, 7);
    ThreeMonths three1 = new ThreeMonths(date: null, booking: null, MembershipId: "", advantages: Advantages4, startingDate: "", endingDate: "", price: 1900.0, isSick

    Advantages[] Advantages5 = new Advantages[2];
```

```

    advantages[2];
    benefitId: "1", name: "Access to Premium Equipment", description: "Equipment");
    benefitId: "2", name: "Personalized Training Sessions", description: "Training");
    null, booking: null, MembershipId: "", advantages: Advantages1, startingDate: "", endingDate: "", price: 1222.0, isSick: true, age: 7);

    advantages[2];
    benefitId: "1", name: "Access to Premium Equipment", description: "Equipment");
    benefitId: "2", name: "Personalized Training Sessions", description: "Training");
    null, booking: null, MembershipId: "", advantages: Advantages2, startingDate: "", endingDate: "", price: 1276.0, isSick: true, age: 7);

    advantages[2];
    benefitId: "1", name: "Access to Premium Equipment", description: "Equipment");
    benefitId: "2", name: "Personalized Training Sessions", description: "Training");
    ths(date: null, booking: null, MembershipId: "", advantages: Advantages3, startingDate: "", endingDate: "", price: 1900.0, isSick: true, age: 7);

    advantages[2];
    benefitId: "1", name: "Access to Premium Equipment", description: "Equipment");
    benefitId: "2", name: "Personalized Training Sessions", description: "Training");
    months(Advantages4, null, "", "", 12.0, true, 7);
    ths(date: null, booking: null, MembershipId: "", advantages: Advantages4, startingDate: "", endingDate: "", price: 1900.0, isSick: true, age: 7);

    advantages[2];
    int numberAdvantages = input.nextInt();
    //add array composition
    Advantages[] adv = new Advantages[numberAdvantages];
    Date newd = new Date();
    Booking newbook = new Booking();
    Membership ml = new Membership(date: newd, booking: newbook, MembershipId: id, advantages: adv, startingDate: subscriptionStartingDate, endingDate: subscriptionEndingDate);
    for (int i = 0; i < adv.length; i++) {
        input.nextLine();
        System.out.print("Enter Advantage id: ");
        String advId = input.next();
        System.out.println("Enter the Advantage name: ");
        String benefitName = input.next();
        input.nextLine();
        System.out.print("Enter the description: ");
        String description = input.nextLine();
        adv[i] = new Advantages(benefitId: advId, name: benefitName, description);
    }

    newd, booking: newbook, MembershipId: id, advantages: adv, startingDate: subscriptionStartingDate, endingDate: subscriptionEndingDate, price: pricePerMonth, isSick: sick, age);
    {
        tage id: ");

        Advantage name: ");
        ();

        escription: ");
        line();
        advId, name: benefitName, description);

        advantages[2];
        new Advantages(benefitId: "1", name: "Access to Premium Equipment", description: "Equipment");
        //ThreeMonths three1 = new ThreeMonths(Advantages4, null, "", "", 12.0, true, 7);
        ThreeMonths three1 = new ThreeMonths(date: null, booking: null, MembershipId: "", advantages: Advantages4, startingDate: "", endingDate: "", price: 1900.0, isSick: true, age: 7);

        Advantages[] Advantages5 = new Advantages[2];
        Advantages5[0] = new Advantages(benefitId: "1", name: "Access to Premium Equipment", description: "Equipment");
        Advantages5[1] = new Advantages(benefitId: "2", name: "Personalized Training Sessions", description: "Training");
        SixMonths six = new SixMonths(freePoolTraining: "free pool training", date: null, booking: null, MembershipId: "", advantages: Advantages5, startingDate: "", endingDate: "", price: 1900.0, isSick: true, age: 7);

        Advantages[] Advantages6 = new Advantages[2];
        Advantages6[0] = new Advantages(benefitId: "1", name: "Access to Premium Equipment", description: "Equipment");
        Advantages6[1] = new Advantages(benefitId: "2", name: "Personalized Training Sessions", description: "Training");
        SixMonths six1 = new SixMonths(freePoolTraining: "free pool training", date: null, booking: null, MembershipId: "", advantages: Advantages6, startingDate: "", endingDate: "", price: 1900.0, isSick: true, age: 7);

        list.add(s: one);
        list.add(s: one1);
        list.add(s: three);
        list.add(s: three1);
        list.add(s: six);
        list.add(s: six1);
    }

```

III.8 exception handling

```
        wf.closeFile();
        System.out.println(x: "All orders saved to the text file orders.txt");
        break;

    case 11://Exit
        System.out.println(x: "Thank you for using our gym system.\nHave a good day...");
        break;
    default:
        System.out.println(x: "Invalid Choice!");
    }
} catch (InputMismatchException ex) {
    System.err.println(x: "Invalid input");
    input.nextLine();
} catch (NullPointerException ex) {
    System.err.println(x: ex);
} catch (ClassCastException ex) {
    System.err.println(x: ex);
} catch (ArrayIndexOutOfBoundsException ex) {
    System.err.println(x: ex);
} catch (Exception ex) {
    System.err.println(x: ex);
}
} while (choice != 11);
}
}
```

III.9 Files

```
case 9://Read from text file
    ReadText rt = new ReadText();
    rt.openTextFile(fileName: "membership.txt");
    rt.readFromFile();
    rt.closeFile();
    break;

case 10://Save/Write to text file
    WriteText wf = new WriteText();
    wf.openTextFile(fileName: "membership.txt");
    if (listt.isEmpty()) {
        System.out.println(x: "there is No orders yet.");
    } else {
        for (Membership ele : listt) {
            wf.writeToFile(m: ele);
        }
    }
    wf.closeFile();
    System.out.println(x: "All orders saved to the text file orders.txt");
    break;
```

```

import java.io.File;
import java.io.FileNotFoundException;
import java.util.NoSuchElementException;
import java.util.Scanner;

public class ReadText {

    private Scanner input;

    public void openTextFile(String fileName) {
        try {
            input = new Scanner(new File(fileName));
        } catch (FileNotFoundException ex) {
            System.err.println("Error opening or creating file.");
            System.err.println(ex);
        }
    }

    public void readFromFile() {
        try {
            while (input.hasNextLine()) {
                System.out.println(input.nextLine());
            }
        } catch (NoSuchElementException ex) {
            System.err.println("File improperly formed.");
            System.err.println(ex);
            System.exit(status: 1);
        } catch (IllegalStateException ex) {
            System.err.println("Error reading from file.");
            System.err.println(ex);
        }
    }

    public void closeFile() {
        if (input != null) {
            input.close();
        }
    }
}

```

```

import java.io.FileNotFoundException;
import java.util.Formatter;
import java.util.FormatterClosedException;

public class WriteText {

    private Formatter output;

    public void openTextFile(String fileName) {
        try {
            output = new Formatter(fileName);
        } catch (SecurityException ex) {
            System.err.println("You do not have write access to this file.");
            System.err.println(ex);
        } catch (FileNotFoundException ex) {
            System.err.println("Error opening or creating file.");
            System.err.println(ex);
        }
    }

    public void writeToFile(Membership m) {
        try {
            output.format(m.toString() + "\n");
            output.format(format: "\n-----\n");
        } catch (FormatterClosedException ex) {
            System.err.println("Error writing to file");
        }
    }
}

```

```

        output.format(format: "\n-----\n");
    } catch (FormatterClosedException ex) {
        System.err.println("Error writing to file");
        System.err.println(ex);
    }
}

public void closeFile() {
    if (output != null) {
        output.close();
    }
}
}

```

IV Main Class

- The **Main** class starts by declaring and initializing three ArrayLists:
- **(list)** from **interface**, **(listt)**, and **(list3)**.
- The **fillList()** method is called to fill the **list** with instances of different types of memberships.
- The **menu()** method is called to display the menu options for the user using do-while
- The user's choice is evaluated using a **switch** statement.
 - Case 1: Adding a Membership.
 - Case 2: Removing a Membership.
 - Case 3: Displaying Memberships.
 - Case 4: search for membership,Creating a new booking,get information from user and checks if it didnt reach max number of booking,cancel booking.
 - Case 5: check if Membership exist.
 - Case 6: Canceling a booking.
 - Case 7: Ordering from the gym restaurant.
 - Case 8: Invoking the GUI JAVA FX.
 - Case 9: Reading from a text file.
 - Case 10: Saving orders to a text file.
 - Case 11: Exiting the program.
- At the end there is try-catch blocks to handle exceptions such as input mismatch, null pointer, class cast, array index out of boundrys.
- The do-while loop continues until the user chooses to exit by selecting the number11.

```

import java.util.ArrayList;
import java.util.InputMismatchException;
import java.util.Scanner;

public class Main {

    static ArrayList<GymPayingSystem> list = new ArrayList<>();
    static ArrayList<Membership> listtt = new ArrayList<>();
    static ArrayList<Booking> list3 = new ArrayList<>();

    public static void fillList() {

        Advantages[] Advantages1 = new Advantages[2];
        Advantages1[0] = new Advantages(benefitId: "1", name: "Access to Premium Equipment", description: "Equipment");
        Advantages1[1] = new Advantages(benefitId: "2", name: "Personalized Training Sessions", description: "Training");
        OneMonth one = new OneMonth(date: null, booking: null, MembershipId: "", advantages: Advantages1, startingDate: "", endingDate: "", price: 1222.0, isSick: true, a

        Advantages[] Advantages2 = new Advantages[2];
        Advantages2[0] = new Advantages(benefitId: "1", name: "Access to Premium Equipment", description: "Equipment");
        Advantages2[1] = new Advantages(benefitId: "2", name: "Personalized Training Sessions", description: "Training");
        OneMonth one1 = new OneMonth(date: null, booking: null, MembershipId: "", advantages: Advantages2, startingDate: "", endingDate: "", price: 1276.0, isSick: true,

        Advantages[] Advantages3 = new Advantages[2];
        Advantages3[0] = new Advantages(benefitId: "1", name: "Access to Premium Equipment", description: "Equipment");
        Advantages3[1] = new Advantages(benefitId: "2", name: "Personalized Training Sessions", description: "Training");
        ThreeMonths three = new ThreeMonths(date: null, booking: null, MembershipId: "", advantages: Advantages3, startingDate: "", endingDate: "", price: 1900.0, isSick:

nPayingSystem> list = new ArrayList<>();
nbership> listtt = new ArrayList<>();
t<Booking> list3 = new ArrayList<>();

fillList() {
vantages1 = new Advantages[2];
= new Advantages(benefitId: "1", name: "Access to Premium Equipment", description: "Equipment");
= new Advantages(benefitId: "2", name: "Personalized Training Sessions", description: "Training");
new OneMonth(date: null, booking: null, MembershipId: "", advantages: Advantages1, startingDate: "", endingDate: "", price: 1222.0, isSick: true, age: 7);

vantages2 = new Advantages[2];
= new Advantages(benefitId: "1", name: "Access to Premium Equipment", description: "Equipment");
= new Advantages(benefitId: "2", name: "Personalized Training Sessions", description: "Training");
new OneMonth(date: null, booking: null, MembershipId: "", advantages: Advantages2, startingDate: "", endingDate: "", price: 1276.0, isSick: true, age: 7);

vantages3 = new Advantages[2];
= new Advantages(benefitId: "1", name: "Access to Premium Equipment", description: "Equipment");
= new Advantages(benefitId: "2", name: "Personalized Training Sessions", description: "Training");
se = new ThreeMonths(date: null, booking: null, MembershipId: "", advantages: Advantages3, startingDate: "", endingDate: "", price: 1900.0, isSick: true, age: 7);

Advantages[] Advantages4 = new Advantages[2];
Advantages4[0] = new Advantages(benefitId: "1", name: "Access to Premium Equipment", description: "Equipment");
Advantages4[1] = new Advantages(benefitId: "2", name: "Personalized Training Sessions", description: "Training");
//ThreeMonths three1 = new ThreeMonths(Advantages4, null, "", "", 12.0, true, 7);
ThreeMonths three1 = new ThreeMonths(date: null, booking: null, MembershipId: "", advantages: Advantages4, startingDate: "", endingDate: "", price: 1900.0, isSick: t

Advantages[] Advantages5 = new Advantages[2];
Advantages5[0] = new Advantages(benefitId: "1", name: "Access to Premium Equipment", description: "Equipment");
Advantages5[1] = new Advantages(benefitId: "2", name: "Personalized Training Sessions", description: "Training");
SixMonths six = new SixMonths(freePoolTraining: "free pool training", date: null, booking: null, MembershipId: "", advantages: Advantages5, startingDate: "", endingDate:

Advantages[] Advantages6 = new Advantages[2];
Advantages6[0] = new Advantages(benefitId: "1", name: "Access to Premium Equipment", description: "Equipment");
Advantages6[1] = new Advantages(benefitId: "2", name: "Personalized Training Sessions", description: "Training");
SixMonths six1 = new SixMonths(freePoolTraining: "free pool training", date: null, booking: null, MembershipId: "", advantages: Advantages6, startingDate: "", endingDat

list.add(e: one);
list.add(e: one1);
list.add(e: three);
list.add(e: three1);
list.add(e: six);
list.add(e: six1);

}

```

```

me:"Access to Premium Equipment", description:"Equipment");
me:"Personalized Training Sessions", description:"Training");
ges4,null , "", "", 12.0,true,7);
booking:null,MembershipId:"",advantages: Advantages4, startingDate:"", endingDate:"", price: 1900.0, isSick: true, age:7);

me:"Access to Premium Equipment", description:"Equipment");
me:"Personalized Training Sessions", description:"Training");
ee pool training",date:null,booking:null,MembershipId:"",advantages: Advantages5,startingDate:"", endingDate:"", price: 2000.0, isSick: true, age:7);

me:"Access to Premium Equipment", description:"Equipment");
me:"Personalized Training Sessions", description:"Training");
ree pool training",date:null,booking:null,MembershipId:"",advantages: Advantages6,startingDate:"", endingDate:"", price: 2000.0, isSick: true, age:7);

```

```

public static void menu() {
    System.out.print("\n1. Add Membership.\n"
        + "2. Remove Membership.\n"
        + "3. Display Membership.\n"
        + "4. New booking.\n"
        + "5. Show Membership.\n"
        + "6. Cancel Membership.\n"
        + "7. Order from gym restuarant \n"
        + "8. GUI.\n"
        + "9. Read From File.\n"
        + "10.Save Orders.\n"
        + "11.Exit.\n"
        + ">> ");
}

public static void main(String[] args) {

    Scanner input = new Scanner(source: System.in);
    int choice = 0;
    fillList();
    System.out.println(x: " WELCOME TO THE GYM SYSTEM ");
    do {
        try {
            menu();
            choice = input.nextInt();
        } catch (Exception e) {}
    } while (choice != 11);

    fillList();
    System.out.println(x: " WELCOME TO THE GYM SYSTEM ");
    do {
        try {
            menu();
            choice = input.nextInt();
            switch (choice) {
                case 1://Add Subscription
                    System.out.print("1. one month Membership\n"
                        + "2. three month Membership\n"
                        + "3. six month Membership\n"
                        + "Select type of the Membership to add >> ");
                    int MembershipType = input.nextInt();

                    if ( MembershipType< 1 || MembershipType > 6) {
                        System.out.println(x: "Invalid choice.");
                    } else {
                        Membership subscription = null;
                        System.out.print(s: "Enter subscription starting date: ");
                        String subscriptionStartingDate = input.next();
                        input.nextLine();
                        System.out.print(s: "Enter subscription ending date: ");
                        String subscriptionEndingDate = input.next();
                        System.out.print(s: "Enter price per month:");
                        double pricePerMonth = input.nextDouble();
                    }
                }
            }
        } catch (Exception e) {}
    } while (choice != 11);
}

```



```

        break;

        default:
            System.out.println(x: "***coach details***");
            System.out.print(s: "Enter coach name: ");
            String coachName = input.nextLine();
            System.out.println(x: "Enter coach mainSpecialization: ");
            String mainSpecialization = input.nextLine();
            System.out.println(x: "Enter coach email: ");
            String coachemail = input.nextLine();
            System.out.println(x: "Enter coach number: ");
            int coachnum = input.nextInt();
            System.out.println(x: "Enter coach address: ");
            String coachadress = input.nextLine();
            Coach coach1=new Coach(speciality: mainSpecialization,name: coachName,new User(email: coachemail,number: coachnum,address:coach

        break;
    }
    list.add(e: subscription);
    System.out.println(x: "membership has been added to the list.");

} //end else
break;

case 2:

intln(x: "***coach details***");
int(s: "Enter coach name: ");
ame = input.nextLine();
intln(x: "Enter coach mainSpecialization: ");
ecialization = input.nextLine();
intln(x: "Enter coach email: ");
mail = input.nextLine();
intln(x: "Enter coach number: ");
= input.nextInt();
intln(x: "Enter coach address: ");
dress = input.nextLine();
new Coach(speciality: mainSpecialization,name: coachName,new User(email: coachemail,number: coachnum,address:coachadress));

```

```

on);
"membership has been added to the list.");

```

```

    case 2:

        System.out.print(s: "Enter membership id to remove: ");
        String membid = input.next();
        boolean removed = false;
        for (int i = 0; i < listt.size(); i++) {
            Membership ele = listt.get(index: i);
            if (ele.getMembershipId().equals(membid)) {
                listt.remove(index: i);
                removed = true;
                break;
            }
        }
        if (removed) {
            System.out.println("Membership with id " + membid + " is deletd");
        } else {
            System.out.println("there is no Membership with id " + membid + " exist");
        }
        break;

        case 3://Display member
        //done
        for (GymPayingSystem ele : list) {
            // Polymorphic methods (Polymorphism)
            System.out.println(e: ele);
        }
    }
}

```

```

case 3://Display member
//done
for (GymPayingSystem ele : list) {
    // Polymorphic methods(Polymorphism)
    System.out.println(x: ele);
    System.out.printf(format: "Membership meal: %.2f%n");
    ele.giveMeal(weight: 0);

    // Downcasting to call specific methods
    System.out.println(x: "-----");
    if (ele instanceof SixMonths) {
        System.out.println(x: ((SixMonths) ele).getFreePoolTraining()); //add an instance in 6 months class or method and call it here
    }
    System.out.println(x: "-----");
}
break;

case 4://New Registration
System.out.printf(format: "Enter membership id: ");
membId = input.next();
//search for Subscription

Membership sub = null;
boolean isFound = false;

for (Membership ele : listt) {

    for (Membership ele : listt) {

        if (ele.getMembershipId().equals(anObject: membId)) {
            sub = ele;
            isFound = true;
            break;
        }
    }
    if (!isFound) {
        System.out.println(x: "Invalid membership id try again later..");
        continue;
    }
    System.out.println(x: "Enter membership starting date:");
    System.out.print(s: "year: ");
    int year = input.nextInt();
    if (year < 2024) {
        System.out.println(x: "wrong year.");
        continue;
    }
    System.out.print(s: "month: ");
    int month = input.nextInt();
    if (month < 1 || month > 12) {
        System.out.println(x: "wrong month.");
        continue;
    }
}

```

```

    }
    System.out.print(s: "day: ");
    int day = input.nextInt();

    if (day < 1 || day > 31) {
        System.out.println(x: "wrong day.");
        continue;
    }
    Date date1=new Date(year,month,day);
    //==get member information for booking==
    System.out.print(s: "Enter your email: ");
    String email = input.next();
    input.nextLine();
    System.out.print(s: "Enter your adress: ");
    String adress = input.nextLine();
    System.out.print(s: "Enter phone number: ");
    int phoneNo = input.nextInt();
    System.out.print(s: "Enter youre name: ");
    String name = input.next();
    User user1= new User(email,number: phoneNo,address:adress);
    Member member1=new Member (name,user: user1);
    Coach C =new Coach();
    Booking b=new Booking(Bookingnum: 1, coach: C,member: member1, maxBooking: 15);

    System.out.print(s: "how many bookings do u want? ");
    int answer = input.nextInt();
    b.setMaxBooking(maxBooking: answer);

    System.out.print(s: "do u want to remove booking? (Yes/No): ");
    String answer2 = input.next();
    if (answer2.endsWith(suffix: "yes") || answer2.endsWith(suffix: "Yes")) {
        b.cancelBooking();
        list3.remove(o: b);
        System.out.println(x: "Your booking has been canceled");

        System.out.println(x: "Your booking has been placed");
    }

    break;

case 5:
    System.out.print(s: "Enter MemberNumber : ");
    int memnum = input.nextInt();
    boolean found = false;
    for (Membership ele : listtt) {
        if (ele.getMembershipId() .equals(anObject: memnum) ) {
            System.out.println(x: ele);
            found = true;
        }
    }
    if (found == false) {
        System.out.println("MemberNumber : " + memnum + " is not found");
    }
    break;

```

```

case 6://Cancel booking
    System.out.print(s: "Enter booking id: ");
    int bookingnum = input.nextInt();
    found = false;
    for (Booking ele : list3) {
        if (ele.getBookingnum() == bookingnum) {
            ele.cancelBooking();
            System.out.println(x: ele);
            found = true;
        }
    }
    if (found == false) {
        System.out.println("booking with number " + bookingnum + " is not found");
    }
    break;

case 7:
    ArrayList<GymRestaurant> GymRestaurant = new ArrayList<>();
    GymRestaurant.add(new GymRestaurant(mealName: "amlond power shake"));
    GymRestaurant.add(new GymRestaurant(mealName: "pasta"));
    GymRestaurant.add(new GymRestaurant(mealName: "crab salad"));
    GymRestaurant.add(new GymRestaurant(mealName: "sugar-free coffee"));

    for (int i = 0; i < GymRestaurant.size(); i++) {
        System.out.println(i + "-" + GymRestaurant.get(index: i));
        System.out.println(x: "-----");
    }
    for (int i = 0; i < GymRestaurant.size(); i++) {
        System.out.println(i + "-" + GymRestaurant.get(index: i));
        System.out.println(x: "-----");
    }
    System.out.print(s: "Enter the item number you want:");
    int index = input.nextInt();
    if (index < 0 || index >= GymRestaurant.size()) {
        System.out.println(x: "invalid number try a valid one !");
        continue;
    }

    GymRestaurant item = GymRestaurant.get(index);

    System.out.println(x: item);

    System.out.println(x: "Thanks for youre order.");

    break;

case 8:

```

```

        case 8://GUI
            //GUI.main(null); //calling static method
            GYMgui.main(null);
            break;
    case 8:

        //GUI
        //GUI.main(null); //calling static method

        break;

    case 9://Read from text file
        ReadText rt = new ReadText();
        rt.openTextFile(fileName: "membership.txt");
        rt.readFromFile();
        rt.closeFile();
        break;

    case 10://Save/Write to text file
        WriteText wf = new WriteText();
        wf.openTextFile(fileName: "membership.txt");
        if (listt.isEmpty()) {
            System.out.println(x: "there is No orders yet.");
        } else {
            for (Membership ele : listt) {
                wf.writeToFile(m: ele);
            }
        }
        wf.closeFile();
        System.out.println(x: "All orders saved to the text file orders.txt");
        break;

        wf.closeFile();
        System.out.println(x: "All orders saved to the text file orders.txt");
        break;

    case 11://Exit
        System.out.println(x: "Thank you for using our gym system.\nHave a good day...");
        break;
    default:
        System.out.println(x: "Invalid Choice!");
    }
} catch (InputMismatchException ex) {
    System.err.println(x: "Invalid input");
    input.nextLine();
} catch (NullPointerException ex) {
    System.err.println(x: ex);
} catch (ClassCastException ex) {
    System.err.println(x: ex);
} catch (ArrayIndexOutOfBoundsException ex) {
    System.err.println(x: ex);
} catch (Exception ex) {
    System.err.println(x: ex);
}
} while (choice != 11);
}
}

```